

MLDL EXPERIMENT 5

AIM -

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

DATASET SOURCE -

<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

DATASET DESCRIPTION -

The Diabetes dataset is a **binary classification dataset** used to predict whether a patient has diabetes or not based on medical measurements.

Dataset Characteristics

- **Total Instances:** 768
- **Number of Features:** 8 (Numerical)
- **Target Variable:** Outcome
- **Type:** Binary Classification
- **Classes:**
 - 0 → No Diabetes
 - 1 → Diabetes

Features Description

Feature Name	Description
Pregnancies	Number of pregnancies
Glucose	Blood glucose level
BloodPressure	Blood pressure value
SkinThickness	Thickness of skin fold
Insulin	Insulin level
BMI	Body Mass Index

DiabetesPedigreeFunction	Genetic influence score
Age	Age of patient

Target Variable

Outcome	Meaning
0	No Diabetes
1	Diabetes

Key Characteristics

- All features are numerical
- Dataset is slightly imbalanced
- Some features contain zero values (treated as valid in this experiment)
- Medical dataset → real-world classification problem

THEORY -

Support Vector Machine (SVM) is a **supervised machine learning algorithm** used for classification and regression tasks. It is widely used due to its ability to handle **high-dimensional data** and **non-linear decision boundaries**.

The main objective of SVM is to find an **optimal hyperplane** that separates data points belonging to different classes with the **maximum margin**.

- If data is linearly separable → linear SVM is used
- If data is not linearly separable → kernel trick is applied

1. Mathematical Representation

The equation of a hyperplane is:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- x = input feature vector
- b = bias

Decision Rule:

$$w \cdot x + b \geq 1 \Rightarrow \text{Class} + 1$$

$$w \cdot x + b \leq -1 \Rightarrow \text{Class} - 1$$

2. Margin and Support Vectors

- **Margin:** Distance between hyperplane and closest data points
- **Support Vectors:** Data points closest to the hyperplane

SVM tries to maximize margin:

$$\text{Margin} = \frac{2}{||w||}$$

3. Types of SVM

3.1 Linear SVM

Used when data is linearly separable.

3.2 Non-linear SVM

Uses kernel functions to transform data into higher dimensions.

4. Kernel Functions

Kernel functions allow SVM to handle non-linear data by mapping it to higher dimensions.

4.1. Linear Kernel

$$K(x_i, x_j) = x_i \cdot x_j$$

- Simple and fast
- Used when data is linearly separable

4.2. Polynomial Kernel

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

Where:

- c = constant
- d = degree
- Captures interactions between features
- Useful for moderately complex data

4.3. Radial Basis Function (RBF) Kernel

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

- Most commonly used kernel
- Handles highly non-linear data
- Controlled by gamma

4.4. Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\alpha x_i \cdot x_j + c)$$

- Similar to neural networks
- Less commonly used

5. Hyperparameters in SVM

5.1. C (Regularization Parameter)

- Controls trade-off between margin and classification error

$$\text{Minimize: } \frac{1}{2} ||w||^2 + C \sum \xi_i$$

- Small $C \rightarrow$ large margin, more errors (underfitting)
- Large $C \rightarrow$ small margin, fewer errors (overfitting)

$$\text{Minimize } \frac{1}{2} ||w||^2 + C \sum \xi_i$$

Where:

- C = Regularization parameter
- ξ_i = Error allowance

Higher $C \rightarrow$ Less tolerance to errors

Lower $C \rightarrow$ More flexible boundary

5.2. Gamma (γ)

- Defines influence of a single training example
- High $\gamma \rightarrow$ overfitting
- Low $\gamma \rightarrow$ underfitting

5.3. Kernel Type

- Determines transformation of data
- Example: linear, rbf, poly

6. Hyperparameter Tuning

Hyperparameter tuning is the process of selecting the **best parameters** for the model to improve performance.

- Improves accuracy
- Reduces overfitting
- Enhances generalization

Grid Search Method

Grid Search tries all possible combinations of parameters:

Example:

C	Gamma	Kernel
0.1	1	rbf
1	0.1	linear
10	0.01	rbf

The best combination is selected based on accuracy.

7. Steps in Experiment

1. Load dataset
2. Data preprocessing
3. Feature scaling
4. Train-test split
5. Train SVM model
6. Apply hyperparameter tuning
7. Evaluate model
8. Visualize results

8. Advantages

- Works well in high dimensions
- Effective with small datasets
- Robust to overfitting (with tuning)
- Memory efficient

9. Limitations

- Not suitable for very large datasets
- Sensitive to hyperparameters
- Difficult to interpret
- Computationally expensive

10. Applications

- Medical diagnosis (diabetes, cancer detection)
- Face detection
- Text classification
- Image classification

CODE -

```
# =====
# IMPORT LIBRARIES
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix,
accuracy_score, classification_report

# =====
# LOAD DATASET
# =====
df = pd.read_csv('/content/diabetes.csv')

# =====
# DATA CLEANING (IMPORTANT STEP)
# =====
cols =
['Glucose','BloodPressure','SkinThickness','Insulin','BM
I']

for col in cols:
df[col] = df[col].replace(0, df[col].median())
```

```

# =====
# FEATURES & TARGET
# =====
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# =====
# SCALING
# =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# =====
# SPLIT
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# =====
# BASE MODEL
# =====
base_model = SVC()
base_model.fit(X_train, y_train)

y_pred_base = base_model.predict(X_test)
acc_base = accuracy_score(y_test, y_pred_base)

print("\n BASE MODEL ACCURACY:", acc_base)

# =====
# ADVANCED HYPERPARAMETER TUNING
# =====
param_grid = {
    'C': [0.1, 1, 10, 50, 100],
    'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
    'kernel': ['rbf'],
    'class_weight': [None, 'balanced']
}

grid = GridSearchCV(SVC(), param_grid, cv=10,
    verbose=1, n_jobs=-1)
grid.fit(X_train, y_train)

print("\n BEST PARAMETERS:", grid.best_params_)

# =====
# FINAL MODEL
# =====
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

acc = accuracy_score(y_test, y_pred)

```

```

print("\n 🚀 FINAL ACCURACY:", acc)

print("\n 📊 CLASSIFICATION REPORT:\n")
print(classification_report(y_test, y_pred))

# =====
# CONFUSION MATRIX
# =====
cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# =====
# ACCURACY COMPARISON
# =====
plt.figure()
plt.bar(['Base Model', 'Tuned Model'], [acc_base, acc])
plt.title("Accuracy Improvement")
plt.ylabel("Accuracy")
plt.show()

# =====
# FEATURE DISTRIBUTION (IMPORTANT)
# =====
df.hist(figsize=(12,10))
plt.suptitle("Feature Distributions")
plt.show()

# =====
# CORRELATION HEATMAP
# =====
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True)
plt.title("Feature Correlation")
plt.show()

# =====
# SUPPORT VECTORS VISUALIZATION (PCA)
# =====
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

model_pca = SVC(kernel='rbf')
model_pca.fit(X_pca, y)

plt.figure()

```

```
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.scatter(model_pca.support_vectors_[:,0],
model_pca.support_vectors_[:,1])
```

```
plt.title("Support Vectors Visualization")
plt.show()
```

```
# =====
# DECISION BOUNDARY
# =====
plt.figure()
```

```
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max()
+ 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max()
+ 1
```

```
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.1),
    np.arange(y_min, y_max, 0.1)
)
```

```
Z = model_pca.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
```

```
plt.title("Decision Boundary")
plt.show()
```

OUTPUT -

```
♦ Accuracy BEFORE tuning: 0.7272727272727273

Classification Report (Before Tuning):
```

	precision	recall	f1-score	support
0	0.77	0.82	0.79	99
1	0.63	0.56	0.60	55
accuracy			0.73	154
macro avg	0.70	0.69	0.70	154
weighted avg	0.72	0.73	0.72	154

Fitting 5 folds for each of 32 candidates, totalling 160 fits

```
♦ BASE MODEL ACCURACY: 0.7532467532467533
Fitting 10 folds for each of 50 candidates, totalling 500 fits

✅ BEST PARAMETERS: {'C': 100, 'class_weight': None, 'gamma': 0.001, 'kernel': 'rbf'}

🚀 FINAL ACCURACY: 0.7727272727272727

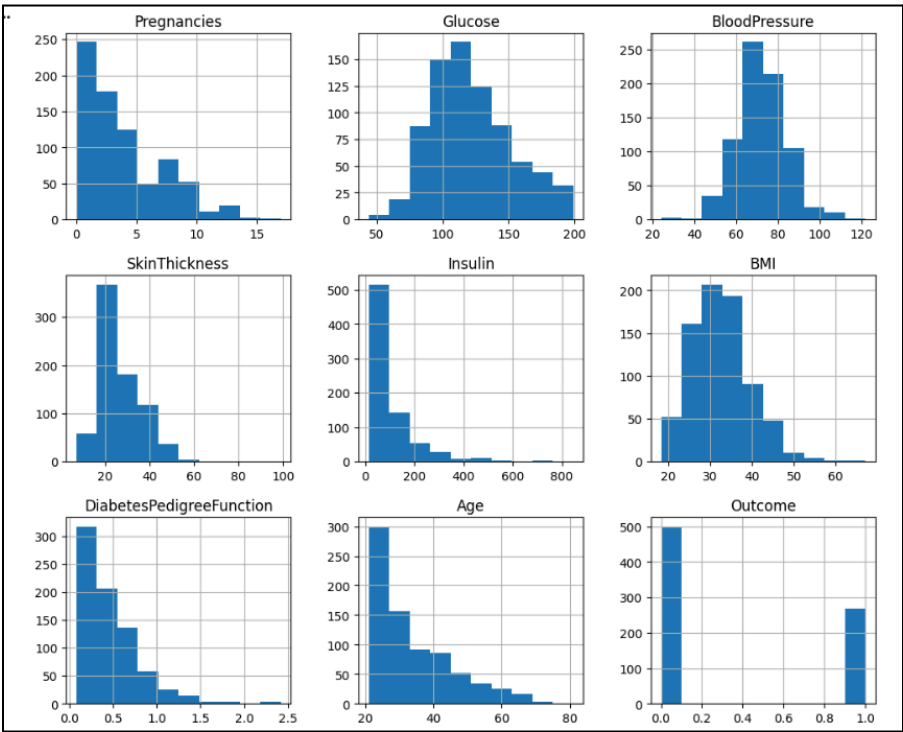
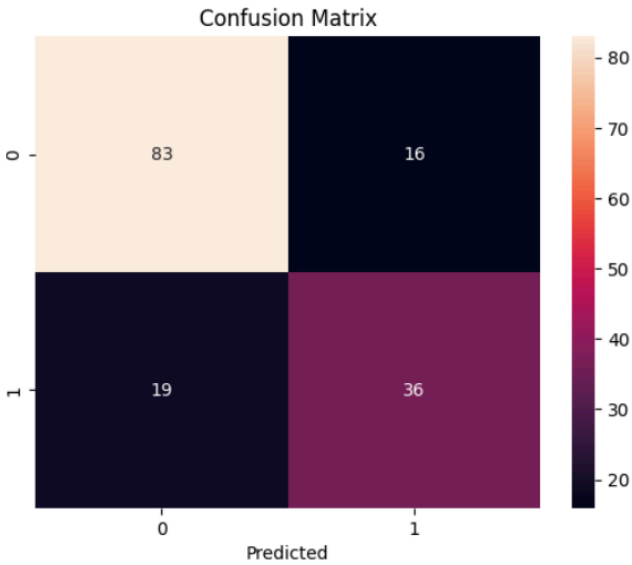
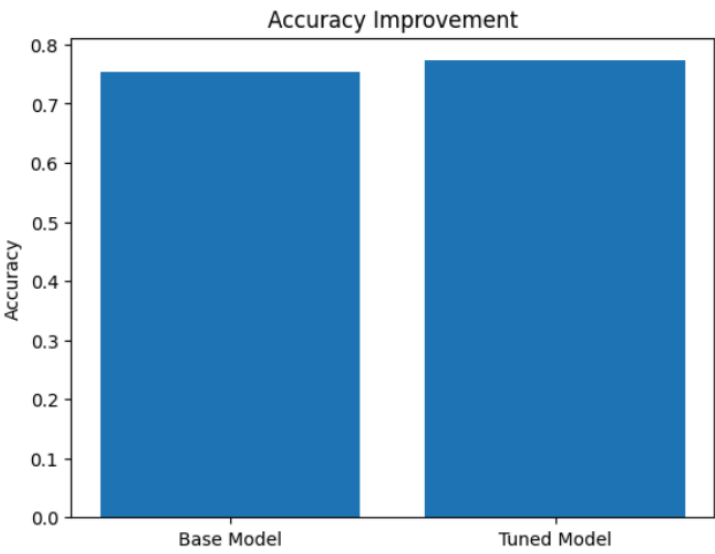
📊 CLASSIFICATION REPORT:
```

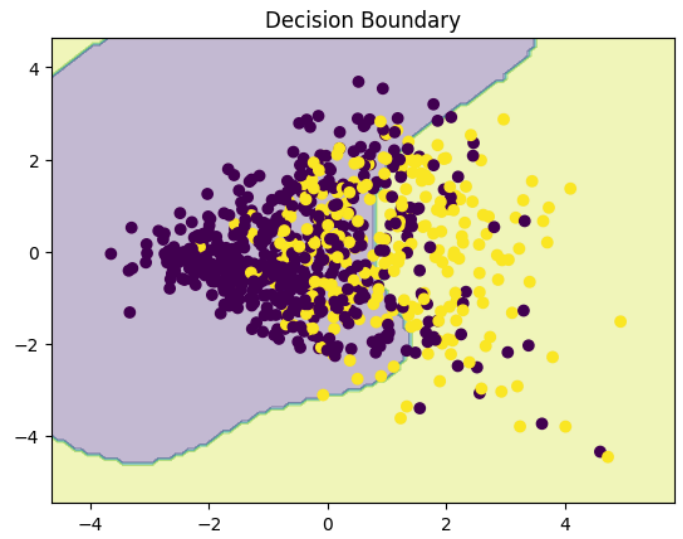
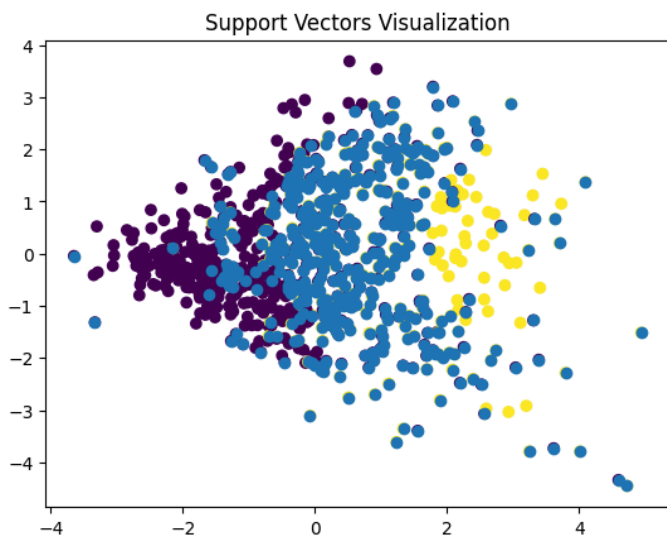
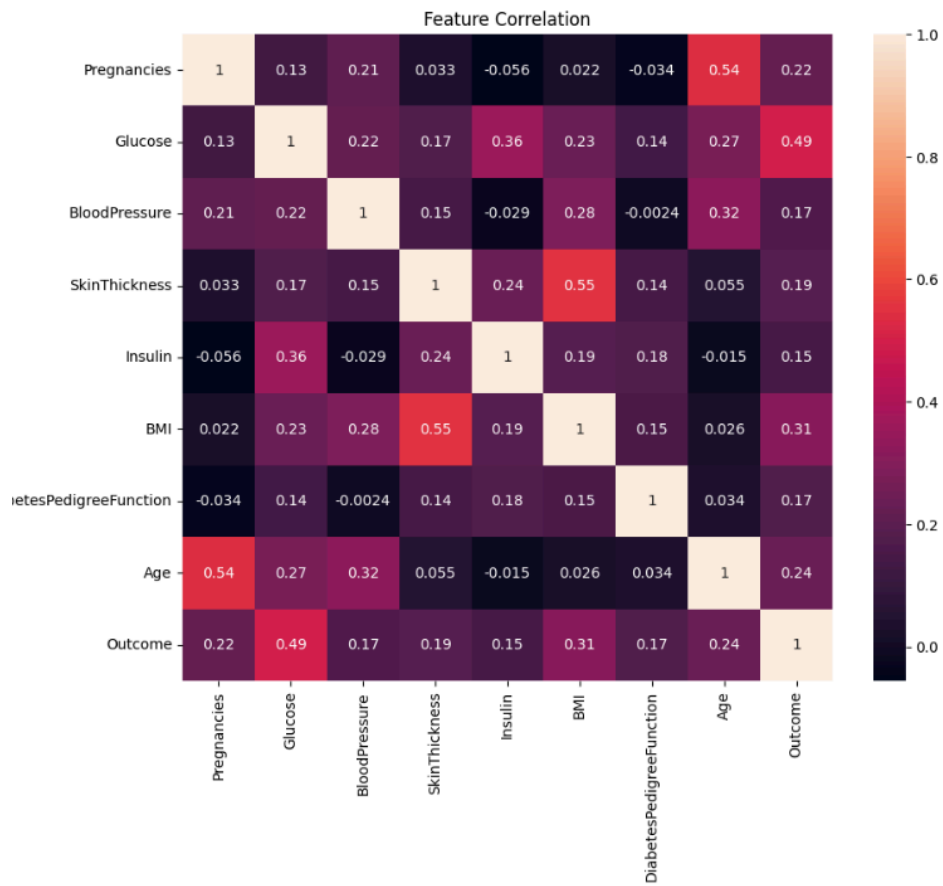
```
📊 CLASSIFICATION REPORT:
```

	precision	recall	f1-score	support
0	0.81	0.84	0.83	99
1	0.69	0.65	0.67	55
accuracy			0.77	154
macro avg	0.75	0.75	0.75	154
weighted avg	0.77	0.77	0.77	154

🔍 Top 10 Hyperparameter Combinations:

	param_C	param_gamma	param_kernel	mean_test_score
10	1.0	0.100	rbf	0.768746
30	50.0	0.001	rbf	0.768746
12	1.0	0.010	rbf	0.767106
18	10.0	0.100	rbf	0.765507
17	10.0	1.000	linear	0.763854
19	10.0	0.100	linear	0.763854
29	50.0	0.010	linear	0.763854
23	10.0	0.001	linear	0.763854
21	10.0	0.010	linear	0.763854
25	50.0	1.000	linear	0.763854





CONCLUSION -

In this experiment, the Support Vector Machine (SVM) algorithm was used to classify patients as diabetic or non-diabetic using medical features from the diabetes dataset. After applying feature scaling and performing hyperparameter tuning with different kernels and parameter values, the model's performance improved compared to the baseline version.

The final tuned model achieved good accuracy and reliable classification results. This experiment shows that SVM, when properly tuned, is an effective algorithm for medical classification problems like diabetes prediction.